

CSE 232 : Data Structures

LAB 05 : LIST

Due Saturday at 5:00 p.m. MST

The fifth programming assignment of this semester is to write a class with the same functionality as the standard template library (STL) `std::list` class.

List

Your list class will be in `list.h` and be named `custom::list`. It will need to support the following operations:

- **Construct:** default constructor, copy constructor, move constructor, non-default empty fill constructor, non-default value fill constructor, range constructor, and initializer list constructor.
- **Assign:** copy assign, move assign, and initializer list assign.
- **Iterator:** begin and end.
- **Access:** front (read/write only), back (read/write only).
- **Insert:** push-front, push-back, and insert. All three have both copy and move versions.
- **Remove:** pop-back, pop-front, and erase.
- **Status:** size and empty.

Iterator

Additionally, we will be creating a list iterator. Only a read/write forward iterator is needed. The following methods will be supported:

- **Construct:** default constructor, copy constructor, non-default construct (so `begin()` and `end()` can properly initialize it).
- **Assign:** copy assign.
- **Compare:** equal-to and not-equal-to.
- **Access:** dereference (read-only and read/write).
- **Arithmetic:** increment (prefix and postfix), decrement (also prefix and postfix).

Three Options

This week, there will be three options.

85 Option : Integer Base Type

This option will require you to implement all the above interfaces, but integer will be the base data-type. As such there is no template parameter. There are three member variables in the `list` class:

```
size_t numElements; // number of elements
Node* pHead;        // pointer to the beginning of the list
Node* pTail;         // pointer to the ending of the list
```

The Node class is a nest class within list. It has three member variables:

```
int data;           // user data
Node * pNext;       // pointer to next node
Node * pPrev;       // pointer to previous node
```

To complete the 85 option, please start with the following files: [232.05.Lab.85.zip](#).

100 Option : Template Base Type

This option will require you to implement all the same interfaces, but a template will be the basetype.

```
template <typename T>
```

There are three member variables:

```
size_t numElements; // number of elements
Node * pHead;        // pointer to the beginning of the list
Node * pTail;        // pointer to the ending of the list
```

The Node class is a nested class and has three member variables:

```
T data;           // user data
Node * pNext;     // pointer to next node
Node * pPrev;     // pointer to previous node
```

To complete the 100 option, please start with the following files: [232.05.Lab.100.zip](#).

115 Option : Allocations

This will require the class to everything from the 100 option. Additionally, the client can specify an allocator as we did in Lab 02. There are two template parameters for this option:

```
template <typename T, typename A = std::allocator<T>>
```

This option has four member variables

```
A alloc;           // use allocator for memory allocation
size_t numElements; // number of elements
Node * pHead;       // pointer to the beginning of the list
Node * pTail;       // pointer to the ending of the list
```

To complete the 115 option, please start with the following files: [232.05.Lab.115.zip](#).

As with last week, the 115 option requires a precise number of allocations and deletions. In order to have more control over this, it is necessary to use an allocator. Please see Appendix I: Allocator in the text book for a detailed description of how this works. There is also a video demonstration here: [Allocators and Spys](#). This explains how the `spy` class works to count allocations, and how `std::allocator` can be use to more precisely control them.

Assignment

Please do the following:

1. Select an option: 85, 100, or 115.
2. Download the code.
3. Implement the data structure with your partner(s).

4. Report how long it took for each individual, and what was the hardest part.
5. Upload your `list.h` code.

Assessment

The grade for your assignment will be based on the following criteria:

	Exceptional 100%	Good 90%	Acceptable 70%	Developing 50%	Missing 0%
Code Quality 35%	The most elegant solution was found for all the methods; there is no room for improvement,	There are no errors in efficiency, maintainability, cohesion, robustness, or redundancy.	There exists one error in efficiency, maintainability, cohesion, robustness, or redundancy.	The code quality is not what is expected for a CSE 232 student.	The code is sloppy or otherwise unprofessional.
Functionality 50%	Passes all the tests	Passes 90% of unit tests	Passes 70% of unit tests	Passes 50% of unit tests	Program fails to compile or no file was submitted
Template Base Type 15%	The 100 or the 115 solution was attempted				The 85 solution was attempted
Allocator 15%	115 solution was attempted				The 85 or the 100 solution was attempted